

Activity 1: Loading a CSV Dataset

Dataset: Iris Dataset

Dataset Link:

<https://archive.ics.uci.edu/dataset/53/iris>

Objectives

- Download a CSV dataset.
- Load it using Pandas.
- Explore its contents.

Exercises

1. Download the dataset.
2. Load it into a Pandas DataFrame.
3. Display
 - first 10 rows
 - last 5 rows
4. Find
 - number of rows
 - number of columns
 - column names
5. Display the data types.
6. Generate summary statistics.
7. Count the number of samples in each class.

CODE:

1.Displaying first 10 values:

```
from google.colab import files
import pandas as pd
```

```
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))
```

```
df.head(10)
```

OUTPUT:

Choose files 4 files

*** **bezdekIris.data**(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
Index(n/a) - 105 bytes, last modified: 29/06/2026 - 100% done
iris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
iris.names(n/a) - 2998 bytes, last modified: 29/06/2026 - 100% done
Saving bezdekIris.data to bezdekIris (5).data
Saving Index to Index (5)
Saving iris.data to iris (5).data
Saving iris.names to iris (5).names

	5.1	3.5	1.4	0.2	Iris-setosa
0	4.9	3.0	1.4	0.2	Iris-setosa
1	4.7	3.2	1.3	0.2	Iris-setosa
2	4.6	3.1	1.5	0.2	Iris-setosa
3	5.0	3.6	1.4	0.2	Iris-setosa
4	5.4	3.9	1.7	0.4	Iris-setosa
5	4.6	3.4	1.4	0.3	Iris-setosa
6	5.0	3.4	1.5	0.2	Iris-setosa
7	4.4	2.9	1.4	0.2	Iris-setosa
8	4.9	3.1	1.5	0.1	Iris-setosa
9	5.4	3.7	1.5	0.2	Iris-setosa

2.Displaying last 5 values:

CODE:

```
from google.colab import files
import pandas as pd

uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

df.tail(5)
```

OUTPUT:

... 4 files

bezdekIris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
Index(n/a) - 105 bytes, last modified: 29/06/2026 - 100% done
iris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
iris.names(n/a) - 2998 bytes, last modified: 29/06/2026 - 100% done
Saving bezdekIris.data to bezdekIris (6).data
Saving Index to Index (6)
Saving iris.data to iris (6).data
Saving iris.names to iris (6).names

	5.1	3.5	1.4	0.2	Iris-setosa
144	6.7	3.0	5.2	2.3	Iris-virginica
145	6.3	2.5	5.0	1.9	Iris-virginica
146	6.5	3.0	5.2	2.0	Iris-virginica
147	6.2	3.4	5.4	2.3	Iris-virginica
148	5.9	3.0	5.1	1.8	Iris-virginica

3. Finding number of rows and columns

CODE:

```
from google.colab import files
import pandas as pd

uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])
```

OUTPUT:

... 4 files

bezdekIris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
Index(n/a) - 105 bytes, last modified: 29/06/2026 - 100% done
iris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
iris.names(n/a) - 2998 bytes, last modified: 29/06/2026 - 100% done
Saving bezdekIris.data to bezdekIris (8).data
Saving Index to Index (8)
Saving iris.data to iris (8).data
Saving iris.names to iris (8).names
Number of Rows: 149
Number of Columns: 5

4. Finding column names

CODE:

```
from google.colab import files
import pandas as pd

uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print("\nColumn Names:")
print(df.columns.tolist())
```

OUTPUT:

```
*** Choose files 4 files
bezdekIris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
Index(n/a) - 105 bytes, last modified: 29/06/2026 - 100% done
iris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
iris.names(n/a) - 2998 bytes, last modified: 29/06/2026 - 100% done
Saving bezdekIris.data to bezdekIris (9).data
Saving Index to Index (9)
Saving iris.data to iris (9).data
Saving iris.names to iris (9).names

Column Names:
['5.1', '3.5', '1.4', '0.2', 'Iris-setosa']
```

5. Displaying datatypes

CODE:

```
from google.colab import files
import pandas as pd

uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print(df.dtypes)
```

OUTPUT:

```
*** Choose files 4 files
bezdekIris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
Index(n/a) - 105 bytes, last modified: 29/06/2026 - 100% done
iris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
iris.names(n/a) - 2998 bytes, last modified: 29/06/2026 - 100% done
Saving bezdekIris.data to bezdekIris (10).data
Saving Index to Index (10)
Saving iris.data to iris (10).data
Saving iris.names to iris (10).names
5.1          float64
3.5          float64
1.4          float64
0.2          float64
Iris-setosa   object
dtype: object
```

6. Generating Summary Statistics & Counting samples

CODE:

```
from google.colab import files
import pandas as pd

uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))

print(df.describe())
print("\nNumber of Samples in Each Class:")
print(df.iloc[:, -1].value_counts())
```

OUTPUT:

```
*** Choose files 4 files
bezdekIris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
Index(n/a) - 105 bytes, last modified: 29/06/2026 - 100% done
iris.data(n/a) - 4551 bytes, last modified: 29/06/2026 - 100% done
iris.names(n/a) - 2998 bytes, last modified: 29/06/2026 - 100% done
Saving bezdekIris.data to bezdekIris (12).data
Saving Index to Index (12)
Saving iris.data to iris (12).data
Saving iris.names to iris (12).names
      5.1      3.5      1.4      0.2
count 149.000000 149.000000 149.000000 149.000000
mean  5.848322  3.054362  3.773826  1.206040
std   0.828594  0.435810  1.760543  0.760354
min   4.300000  2.000000  1.000000  0.100000
25%   5.100000  2.800000  1.600000  0.300000
50%   5.800000  3.000000  4.400000  1.300000
75%   6.400000  3.300000  5.100000  1.800000
max   7.900000  4.400000  6.900000  2.500000

Number of Samples in Each Class:
Iris-setosa
Iris-versicolor    50
Iris-virginica     50
Iris-setosa        49
Name: count, dtype: int64
```

Activity 2: Handling Missing Values

Dataset: Titanic Dataset

Dataset Link:

<https://www.kaggle.com/c/titanic/data>

Objectives

Learn preprocessing techniques.

Exercises

1. Load the dataset.
2. Identify columns containing missing values.
3. Calculate the percentage of missing values.
4. Replace missing numerical values with the median.
5. Replace missing categorical values with the mode.

6. Remove duplicate rows.
7. Verify that no missing values remain.

1.Columns with missing values:

CODE:

```
# Load dataset
url =
"https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

print("\nColumns with Missing Values:")
missing = df.isnull().sum()
print(missing[missing > 0])
```

OUTPUT:

```
***
Columns with Missing Values:
Age          177
Cabin        687
Embarked      2
dtype: int64
```

2.Calculating percentage of missing values

CODE:

```
print("\nPercentage of Missing Values:")
missing_percent = (df.isnull().sum() / len(df)) * 100
print(missing_percent[missing_percent > 0])
```

OUTPUT:

```
***
Percentage of Missing Values:
Age          19.865320
Cabin        77.104377
Embarked      0.224467
dtype: float64
```

3.Replacing numerical values with median

CODE:

```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

```
for col in num_cols:  
    df[col] = df[col].fillna(df[col].median())
```

4.Replacing categorical values with mode

CODE:

```
cat_cols = df.select_dtypes(include=['object']).columns
```

```
for col in cat_cols:  
    df[col].fillna(df[col].mode()[0])
```

5. Remove duplicate rows

CODE:

```
# Remove duplicate rows
```

```
duplicates = df.duplicated().sum()  
print("\nDuplicate Rows:", duplicates)
```

```
df.drop_duplicates(inplace=True)
```

```
# Verify no missing values remain
```

```
print("\nMissing Values After Cleaning:")  
print(df.isnull().sum())
```

```
# Final dataset information
```

```
print("\nDataset Shape After Cleaning:")  
print(df.shape)
```

OUTPUT:

...

Duplicate Rows: 0

Missing Values After Cleaning:

PassengerId 891

Survived 891

Pclass 891

Name 0

Sex 0

Age 891

SibSp 891

Parch 891

Ticket 0

Fare 891

Cabin 0

Embarked 0

dtype: int64

Dataset Shape After Cleaning:

(891, 12)

Activity 3: Feature Encoding

Dataset: Adult Income Dataset

Dataset Link:

<https://archive.ics.uci.edu/dataset/2/adult>

Exercises

1. Load the dataset.
2. Identify categorical and numerical attributes.
3. Apply Label Encoding to binary features.
4. Apply One-Hot Encoding to multi-class features.
5. Compare the dataset dimensions before and after encoding.

1. Identifying categorical and numerical attributes

CODE:

```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

```
cat_cols = df.select_dtypes(include=['object']).columns
```

```
print("Numerical Attributes:")
print(list(num_cols))

print("\nCategorical Attributes:")
print(list(cat_cols))
```

OUTPUT:

```
*** Numerical Attributes:
    ['age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']

    Categorical Attributes:
    ['workclass', 'education', 'marital-status', 'occupation', 'relationship', 'race', 'sex', 'native-country', 'income']
```

2. Apply Label Encoding to Binary features

CODE:

```
from sklearn.preprocessing import LabelEncoder

print("Shape before encoding:", df.shape)
le = LabelEncoder()

for col in cat_cols:
    if df[col].nunique() == 2:
        df[col] = le.fit_transform(df[col])
multi_cols = [col for col in cat_cols if df[col].nunique() > 2]

df = pd.get_dummies(df, columns=multi_cols)

print("Shape after encoding:", df.shape)
print(df.head())
```

OUTPUT:

```
*** Shape before encoding: (48842, 15)
    Shape after encoding: (48842, 111)
   age  fnlwgt  education-num  sex  capital-gain  capital-loss  \
0   39   77516             13    1           2174             0
1   50   83311             13    1              0             0
2   38  215646              9    1              0             0
3   53  234721              7    1              0             0
4   28  338409             13    0              0             0
```

	hours-per-week	workclass_?	workclass_Federal-gov	workclass_Local-gov	\
0	40	False	False	False	
1	13	False	False	False	
2	40	False	False	False	
3	40	False	False	False	
4	40	False	False	False	

	...	native-country_Taiwan	native-country_Thailand	\
0	...	False	False	
1	...	False	False	
2	...	False	False	
3	...	False	False	
4	...	False	False	

	native-country_Trinidad&Tobago	native-country_United-States	\
0	False	True	
1	False	True	
2	False	True	
3	False	True	
4	False	False	

	native-country_Vietnam	native-country_Yugoslavia	income_<=50K	\
0	False	False	True	
1	False	False	True	
2	False	False	True	
3	False	False	True	
4	False	False	True	

	income_<=50K.	income_>50K	income_>50K.
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False

[5 rows x 111 columns]

1. **Dataset:** Wine Dataset

2. **Dataset Link:**

<https://archive.ics.uci.edu/dataset/109/wine>

3. **Exercises**

4. Load the dataset.

5. Compute the minimum and maximum value of each feature.

6. Apply Min-Max Scaling.

7. Apply Standardization.

1.Loading dataset

CODE:

```
from ucimlrepo import fetch_ucirepo

# Fetch dataset
wine = fetch_ucirepo(id=109)

# Features and target
X = wine.data.features
y = wine.data.targets

# Metadata
print(wine.metadata)

# Variable information
print(wine.variables)
```

2.Find Minimum and Maximum Value of every feature

CODE:

```
print("Minimum value of each feature:")
print(X.min())

print("\nMaximum value of each feature:")
print(X.max())
```

OUTPUT:

```
... Minimum value of each feature:
Alcohol          11.03
Malicacid         0.74
Ash              1.36
Alcalinity_of_ash 10.60
Magnesium        70.00
Total_phenols     0.98
Flavanoids        0.34
Nonflavanoid_phenols 0.13
Proanthocyanins   0.41
Color_intensity   1.28
Hue              0.48
0D280_0D315_of_diluted_wines 1.27
Proline          278.00
dtype: float64
```

```

Maximum value of each feature:
Alcohol          14.83
Malicacid         5.80
Ash              3.23
Alcalinity_of_ash 30.00
Magnesium        162.00
Total_phenols     3.88
Flavanoids        5.08
Nonflavanoid_phenols 0.66
Proanthocyanins   3.58
Color_intensity   13.00
Hue              1.71
0D280_0D315_of_diluted_wines 4.00
Proline          1680.00
dtype: float64

```

3.Applying Min-Max Scaling

CODE:

```

from sklearn.preprocessing import MinMaxScaler
import pandas as pd

# Create scaler
minmax_scaler = MinMaxScaler()

# Apply scaling
X_minmax = pd.DataFrame(
    minmax_scaler.fit_transform(X),
    columns=X.columns
)

print("Min-Max Scaled Data:")
print(X_minmax.head())

```

OUTPUT:

```

... Min-Max Scaled Data:
   Alcohol  Malicacid  Ash  Alcalinity_of_ash  Magnesium  Total_phenols  \
0  0.842105  0.191700  0.572193      0.257732   0.619565   0.627586
1  0.571053  0.205534  0.417112      0.030928   0.326087   0.575862
2  0.560526  0.320158  0.700535      0.412371   0.336957   0.627586
3  0.878947  0.239130  0.609626      0.319588   0.467391   0.989655
4  0.581579  0.365613  0.807487      0.536082   0.521739   0.627586

```

	Flavanoids	Nonflavanoid_phenols	Proanthocyanins	Color_intensity \
0	0.573840	0.283019	0.593060	0.372014
1	0.510549	0.245283	0.274448	0.264505
2	0.611814	0.320755	0.757098	0.375427
3	0.664557	0.207547	0.558360	0.556314
4	0.495781	0.490566	0.444795	0.259386

	Hue	0D280_0D315_of_diluted_wines	Proline
0	0.455285	0.970696	0.561341
1	0.463415	0.780220	0.550642
2	0.447154	0.695971	0.646933
3	0.308943	0.798535	0.857347
4	0.455285	0.608059	0.325963

4.Applying standardization

CODE:

```
from sklearn.preprocessing import StandardScaler
```

```
# Create scaler
```

```
standard_scaler = StandardScaler()
```

```
# Apply standardization
```

```
X_standard = pd.DataFrame(
    standard_scaler.fit_transform(X),
    columns=X.columns
)
```

```
print("Standardized Data:")
```

```
print(X_standard.head())
```

OUTPUT:

```
*** Standardized Data:
```

	Alcohol	Malicacid	Ash	Alcalinity_of_ash	Magnesium	Total_phenols \
0	1.518613	-0.562250	0.232053	-1.169593	1.913905	0.808997
1	0.246290	-0.499413	-0.827996	-2.490847	0.018145	0.568648
2	0.196879	0.021231	1.109334	-0.268738	0.088358	0.808997
3	1.691550	-0.346811	0.487926	-0.809251	0.930918	2.491446
4	0.295700	0.227694	1.840403	0.451946	1.281985	0.808997

	Flavanoids	Nonflavanoid_phenols	Proanthocyanins	Color_intensity \
0	1.034819	-0.659563	1.224884	0.251717
1	0.733629	-0.820719	-0.544721	-0.293321
2	1.215533	-0.498407	2.135968	0.269020
3	1.466525	-0.981875	1.032155	1.186068
4	0.663351	0.226796	0.401404	-0.319276

	Hue	0D280_0D315_of_diluted_wines	Proline
0	0.362177	1.847920	1.013009
1	0.406051	1.113449	0.965242
2	0.318304	0.788587	1.395148
3	-0.427544	1.184071	2.334574
4	0.362177	0.449601	-0.037874

Activity 5: Data Splitting

Use any dataset from Activities 1–8.

Exercises

1. Split the dataset into
 - 70% Training
 - 15% Validation
 - 15% Testing
2. Verify the number of samples in each subset.
3. Ensure the class distribution is maintained using stratified sampling.

1.Splitting Dataset into Training, Validation and Testing

CODE

```
from ucimlrepo import fetch_ucirepo
from sklearn.model_selection import train_test_split

# Fetch the Wine dataset
wine = fetch_ucirepo(id=109)

# Features and target
X = wine.data.features
y = wine.data.targets
X_train, X_temp, y_train, y_temp = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp,
    y_temp,
```

```
test_size=0.50,  
random_state=42,  
stratify=y_temp  
)  
print("Training Set:", len(X_train))  
print("Validation Set:", len(X_val))  
print("Testing Set:", len(X_test))
```

OUTPUT

```
*** Training Set: 124  
Validation Set: 27  
Testing Set: 27
```

2. Verifying Number of samples in each subset

CODE

```
print("Training Class Distribution:")  
print(y_train.value_counts())  
  
print("\nValidation Class Distribution:")  
print(y_val.value_counts())  
  
print("\nTesting Class Distribution:")  
print(y_test.value_counts())
```

OUTPUT

*** Training Class Distribution:

class

2 50

1 41

3 33

Name: count, dtype: int64

Validation Class Distribution:

class

2 10

1 9

3 8

Name: count, dtype: int64

Testing Class Distribution:

class

2 11

1 9

3 7

Name: count, dtype: int64